

Python Topic 12 — Modules & Imports

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert

Layer 1 — New to Python

A **module** is just a Python file (`.py`). You use code from one file inside another by **importing** it — like `import` / `require` in JavaScript.

```
# math is a built-in module
import math

math.sqrt(16)    # 4.0
math.pi          # 3.14159...
```

You `import` the module, then use its functions with a dot: `module.function()` . Or import just the parts you need:

```
from math import sqrt, pi

sqrt(16)    # 4.0 → no "math." prefix needed now
pi          # 3.14159...
```

Layer 2 — Intermediate (real code + how it works)

The import styles

```
import math                # whole module → math.sqrt()
from math import sqrt      # one name → sqrt()
from math import sqrt, pi, cos # multiple names
from math import *         # everything (⚠ avoid – pollutes namespace)
import math as m           # alias → m.sqrt()
from datetime import datetime as dt # alias a name
```

JS comparison

```
// JS
import { sqrt } from './math.js';
import * as math from './math.js';
import math from './math.js';    // default export
```

```
# Python (no "default export" concept)
from math import sqrt
import math
```

Importing your own files

```
# file: helpers.py
def greet(name):
    return f"Hello, {name}"
PI = 3.14159

# file: main.py
from helpers import greet, PI
print(greet("Akshay")) # Hello, Akshay
```

The module name is the **filename without .py** — `helpers.py` → `import helpers` .

Common standard-library modules

```
import os          # file paths, environment variables
import sys         # system/interpreter stuff
import json        # JSON parsing (recall Topic 7)
import datetime    # dates and times
import random      # random numbers
import re          # regular expressions
from pathlib import Path # modern file path handling
from collections import Counter, defaultdict
```

Layer 3 — Expert (internals & gotchas)

1. `if __name__ == "__main__":` — run-directly vs imported

A file can be used two ways, and this line makes it behave differently in each. Every file gets a hidden `__name__` variable:

How the file is used	Value of <code>__name__</code>
Run directly (<code>python myfile.py</code>)	<code>"__main__"</code>
Imported (<code>import myfile</code>)	<code>"myfile"</code> (the filename)

Without the guard, test code runs **just from importing** — usually unwanted:

```
# calculator.py
def add(a, b):
    return a + b
print("Testing:", add(2, 3))    # ← runs even when imported 🤔
```

The fix — wrap the "run directly" code:

```
# calculator.py
def add(a, b):
    return a + b

if __name__ == "__main__":
    print("Testing:", add(2, 3))    # only when run directly
```

```
python calculator.py    # __name__ == "__main__" → the block runs
import calculator       # __name__ == "calculator" → block skipped
```

✅ **Real use:** lets one file be both a **reusable library** (import the functions) AND a **runnable script** (run it to test/execute). You'll see it at the bottom of nearly every script.

2. Packages & the actual use of `__init__.py`

`__init__.py` marks a **folder** as an importable **package**, and optionally sets up what happens on import.

```
myproject/
├── utils/
│   ├── __init__.py    ← makes "utils" an importable package
│   ├── strings.py
│   └── numbers.py
└── main.py
```

```
# main.py – possible because utils/ has __init__.py
from utils.strings import clean_text
from utils.numbers import round_up
```

It can be **empty** (its presence alone marks the package). But it also *runs* on first import, so you can re-export for cleaner imports:

```
# utils/__init__.py
from .strings import clean_text
from .numbers import round_up
```

```
# main.py – now cleaner
from utils import clean_text, round_up
```

i JS analogy: `__init__.py` is roughly a folder's `index.js` — it defines the folder as a module and can re-export things. (Python 3.3+ allows packages without it, but including it is clearest and needed for re-export setup.)

3. Modules are cached — run once per program

The **first** time *any* file imports a module, Python finds it, **runs it top to bottom**, and stores it in a cache (`sys.modules`). Every later import — **from anywhere in the program** — reuses the cached version and does **not** re-run the file.

```
# config.py
print("⚙️ config.py is being loaded!")
SETTING = "production"

# main.py
import config      # first import overall → runs config.py ONCE
import a           # a.py imports config → uses cache, doesn't re-run
import b           # b.py imports config → uses cache, doesn't re-run

# Output of `python main.py`:
# ⚙️ config.py is being loaded! ← printed only ONCE
```

i Scope of the cache: it's **program-wide** (across all files), keyed by module name, for one run. Restart the program and the file runs fresh on first import again. This is why edits to a module don't show up in a running REPL/Jupyter until you reload or restart.

4. The circular import trap

"Circular" describes the **files depending on each other** — not the functions. If `a` needs `b` and `b` needs `a`, they form a loop and neither can finish loading.

```
# a.py
from b import func_b    # a imports b

# b.py
from a import func_a    # b imports a → 💥 circular import error
```



Why it breaks: importing runs a file top-to-bottom. Running `a.py` hits `import b` → starts `b.py` → hits `import a` → returns to a **half-loaded** `a.py` where `func_a` isn't defined yet → `ImportError`. The function names don't matter; the mutual file dependency does.

✓ **Fixes:**

- **Shared third module** — move common code to `shared.py` ; both import from it (`a → shared` , `b → shared`). No loop.
- **Deferred import** — move the import *inside* the function so it runs after both files finish loading.
- **Merge** the two modules if they're that tightly coupled.

```
# Deferred import fix (b.py)
def func_b():
    from a import func_a      # imported when CALLED, not at load time
    return func_a()
```

5. Absolute vs relative imports

```
# absolute (preferred – clear and unambiguous)
from myproject.utils.strings import clean_text

# relative (within a package – the dot means "current package")
from .strings import clean_text      # . = same package
from ..config import settings       # .. = parent package
```

6. `__pycache__` & the search path

On import, Python compiles the module to bytecode and caches it in `__pycache__` / (`.pyc` files) for faster loads — add `__pycache__` / to `.gitignore` (like `node_modules`). Import lookup order:

```
import sys
print(sys.path)      # current dir → venv site-packages → standard library
```

A failing `ModuleNotFoundError` is usually a venv/path issue or a typo in the name.

Interview Q&A Cards

Q: What does `if __name__ == "__main__":` do?

A: Its block runs only when the file is executed directly (`__name__` is `"__main__"`), and is skipped when the file is imported (`__name__` is the module name). Lets a file be both a library and a script.

Q: What is `__init__.py` for?

A: It marks a folder as an importable package. It can be empty, or it can run setup/re-export code on first import (like a folder's `index.js`).

Q: Does importing a module run it every time?

A: No. A module runs once, on its first import; the result is cached in `sys.modules`. Every later import anywhere in the program reuses the cache. Restarting the program re-runs it once more.

Q: What is a circular import and how do you fix it?

A: Two modules importing each other form a dependency loop, so one is used while still half-loaded → ImportError. Fix by moving shared code to a third module, deferring the import inside a function, or merging the modules.

🔗 **Memory hook:** Module = file, package = folder (marked by `__init__.py`). `if __name__ == "__main__"` = "only when run directly." A module runs once per program, then it's cached. Circular = files pointing at each other in a loop.

✅ **One-liner for interviews:** "A module is a `.py` file; a package is a folder with modules. Import with `import x` or `from x import y`. `if __name__ == '__main__'` lets a file be both importable and runnable. Modules are cached and run once. Prefer absolute imports; watch for circular imports."